High Performance Linear Algebra and Optimization within C++

Casey Hawthorne

UT EID: ceh4443

TACC User: casyeh

ceh4443@my.utexas.edu

December 2025

Contents

1 Introduction **2**

1.1 Linear Algebra as a Subject . . . 2

1.2 Primary Challenges . . . 2

1.3 Mathematical Preliminaries and Basic Optimization . . . 2

1.4 Matrix Multiplication Permutations . . . 3

2 class: Matrix, Implementing Optimization **4**

2.1 Handling Matrix Storage . . . 5

2.2 Implementation and Optimization . . . 6

2.3 Setting Up for Greater Optimization . . . 7

2.3.1 Experimenting via Simple Operations . . . 7

2.3.2 Debugging and Cache Optimization . . . 8

2.4 Implementation of Sub-matrices and their Methods . . . 10

2.4.1 Sub-matrix Construction and Verification . . . 10

2.4.2 Implementation Through Various Methods . . . 11

2.4.3 Optimization of Sub-Matrix Methods via Recursion . . . 13

3 Testing Optimization: Performance Benchmarks Across Variations **15**

3.1 Base Multiplication Correctness . . . 15

3.2 Performance Optimization . . . 16

3.3 Discussion: Memory Reuse and Efficiency . . . 18

3.4 Recursion and its Diminishing Returns . . . 19

4 Conclusion **20**

5 Appendix **22**

5.1 A: Code Snippets . . . 22

1 Introduction

1.1 Linear Algebra as a Subject

By Definition: The branch of mathematics that deals with vector spaces and the linear mappings (or transformations) between these spaces. Linear Algebra forms the backbone of various fields in computer science, including data science, machine learning, quantum computing, and cryptography.

In regards to its applications within computation,, Linear Algebra is employed on a scale far beyond what is typically encountered in manual mathematical practice (i.e., small, elementary matrices). In real-world computation—ranging from computer graphics and image processing to machine learning and neural networks—the size of matrices, the complexity of calculations, and the computational resources required are substantially greater than what is usually addressed in standard Linear Algebra or introductory computation courses.

Due to this, a variety of optimization strategies are necessary to reduce runtime, energy consumption, and memory usage. Many of these optimizations begin with even the most fundamental matrix operations, as will be explored in this paper.

1.2 Primary Challenges

Our main goal in this project will be to take the naive implementation of matrix multiplication, being a triple nested loop IJK method, and optimize it into something far more efficient and memory friendly that is flexible enough to be applicable in real world scenarios.

1.3 Mathematical Preliminaries and Basic Optimization

In C++, matrices are stored in row-major order, meaning that consecutive elements of a row are contiguous in memory. In typical computation, we often come across the **IJK** method of multiplication, which is the standard triple-nested loop algorithm for multiplying matrices A and B to get C, where the loops iterate in the order of **I** (rows of A/C), **J** (columns of B/C), and **K** (inner dimension) to calculate the following;

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \times B_{kj}$$

coded as follows;

Naive IJK Multiplication

```
        for(int i=0;i<4;i++)
            for(int j=0;j<4;j++){
                C_ijk.at(i,j)=0;
                for(int k=0;k<4;k++)
                    C_ijk.at(i,j) += A_small.at(i,k)*B_small.at(k,j);
        }
```

In the **IJK** order, the innermost loop iterates over the columns of matrix B, which are not contiguous in memory. This leads to frequent cache misses, slowing down the computation

However, we are able to experiment with this method and create slightly more memory efficient and faster permutations of the same algorithm.

1.4 Matrix Multiplication Permutations

By reordering the nested loop from IJK, to KIJ or IKJ, we produce code for a faster, more memory efficient matrix multiplication algorithm. This is shown in the following code snippet;

Optimized IKJ Multiplication

```
    void MatMult(const Matrix& B, Matrix& C) const {
        int M = rows(), N=B.cols(), K=cols();
        for(int i=0;i<M;i++) for(int j=0;j<N;j++) C.at(i,j)=0;
        for(int i=0;i<M;i++) for(int k=0;k<K;k++){
            double aik = at(i,k);
            for(int j=0;j<N;j++) C.at(i,j)+=aik*B.at(k,j);
        }
    }
```

Here, the **IKJ** ordering iterates over the columns of the output matrix C in the innermost loop while keeping the elements of the matrix B's row and the current element of matrix A in the registers. This ensures that the memory accesses are contiguous, allowing values of A to be reused efficiently. As a result, **IKJ** ordering is significantly faster in practice, even though the total number of arithmetic operations remains the same.

This is known as **"outer product based"** because each iteration of the middle loop (over k) computes an outer product of a column of B and a row of A and adds it in.

We may test the accuracy of the **IKJ** against the **IJK** method with the following matrices;

$$A_{\text{small}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \qquad B_{\text{small}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Which output the following for each respective method,

Reference vs Optimized Output

```
C_ref (reference):
 90.000000100.000000110.000000120.000000
202.000000228.000000254.000000280.000000
314.000000356.000000398.000000440.000000
426.000000484.000000542.000000600.000000

C_opt (optimized IKJ):
 90.000000100.000000110.000000120.000000
202.000000228.000000254.000000280.000000
314.000000356.000000398.000000440.000000
426.000000484.000000542.000000600.000000

IKJ equals ref? YES
```

(See Appendix.A, Main)

2 class: Matrix, Implementing Optimization

Another method for multiplying matrices is via block partitioning, which splits matrices A, B, C, be split on a 2x2 block form as such

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

to find

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22}. \end{aligned}$$

We will begin our implementation of this by first creating a class Matrix;

Class Matrix

```
    class Matrix {
public:
    size_t n;                       // assume square matrix
    std::vector<double> data;       // row-major storage
```

and implementing a traditional matrix multiplication method;

```
Matrix Matrix::MatMultTraditional(const Matrix& B) const {
    assert(n == B.n);
    Matrix C(n);

    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < n; j++) {
            double sum = 0.0;
            for (size_t k = 0; k < n; k++)
                sum += (*this)(i, k) * B(k, j);
            C(i, j) = sum;
        }

    return C;
}
```

The traditional method will take the dot products of elements from the row of the first matrix and the columns of the second. However, we may optimize this by altering this algorithm to be recursive, effectively implementing block partitioning. To do this, we will need to add sub-matrix handling, which includes extracting a sub-matrix, and then writing it back into a surrounding matrix

The code for doing this can be written as follows;

Block Partitioning Methods

```
// Extract k k block from (r0, c0)
Matrix Matrix::submatrix(size_t r0, size_t c0, size_t k) const {
    Matrix M(k);
    for (size_t i = 0; i < k; i++)
        for (size_t j = 0; j < k; j++)
            M(i, j) = (*this)(r0 + i, c0 + j);
    return M;
}

// Write submatrix M into current matrix at (r0, c0)
void Matrix::write_submatrix(size_t r0, size_t c0, const Matrix& M) {
    size_t k = M.n;
    for (size_t i = 0; i < k; i++)
        for (size_t j = 0; j < k; j++)
            (*this)(r0 + i, c0 + j) = M(i, j);
}
```

These routines allow our class to handle and partition a matrix into smaller more manageable sub-matrices. However, repeatedly copying sub-matrices can be far from efficient as it will be less efficient with larger matrices, due to struggles with memory allocation.

2.1 Handling Matrix Storage

Above, you saw the idea of block algorithms, which requires taking sub-matrices. However, copying elements into a new array proves to be inefficient, so we want the sub-matrix to correspond to a sub-array.

However, only a sub-matrix that consists of a sequence of columns is contiguous. Thus, the formula i + j · M for the location of element (i, j) is no longer correct if the matrix is a sub-block of a larger matrix.

To address this, linear algebra software describes a sub-matrix with three parameters; M, N, LDA, where 'LDA' stands for 'leading dimension of A' as illustrated in the figure below

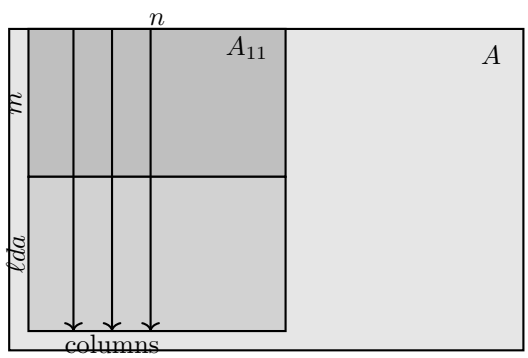

Figure 1: Block structure of a matrix showing sub-matrix A_{11} and dimensions.

Now that we have newly defined parameters, we will need to adjust our formula to find element (i, j) to fit with the newly added LDA

This formula is given by;

$$(r_{offset} + i) + (c_{offset} + j) \cdot \text{LDA}$$

Where i, and j, represent the row and column indices of the sub-matrix, while r_{offset} and c_{offset} represent the row and column offsets of the row and column offsets of the sub-matrix inside the parent matrix.

2.2 Implementation and Optimization

Implementation-wise we come across a limitation of std::vector for storage. It is not possible to take sub-arrays with std::vector, since C++ insists that a vector has its own storage. We overcome this by using span

Now we have top level matrices that store a vector¡double¿, and sub-matrices that store a span¡double¿. We can also adopt the following idiom to create a vector at the top level and then create matrices from its memory.

From 53.2. Matrix Storage, PAGE 543 of Introduction to Scientific Programming in C++23/Fortran2018 by Victor Eijkhout

```
// gemm/lapack alloc.cpp
// example values for M,LDA,N
M = 2; LDA = M+2; N = 3;
// create a vector to contain the data
vector<double> one_data(LDA*N,1.);
// create a matrix using the vector data
Matrix one(M,LDA,N,one_data.data());
```

We will begin by updating our Matrix class to include the constructor

```
Matrix(int m, int n, int lda, double* subptr) : M(m), N(n), LDA(lda), ptr(subptr)
↪ {}
```

And private data members

```
int m_, n_, lda_;
std::vector<double> storage;   // own the memory
std::span<double> data_;       // view into storage
```

As well as the following method to allow us to safely access stored data members.

```
double& at(int i, int j) {
    assert(i>=0 && i<M && j>=0 && j<N);
    return data[i*LDA + j];
}
const double& at(int i, int j) const {
    assert(i>=0 && i<M && j>=0 && j<N);
    return data[i*LDA + j];
}
```

However, as with much of our code so far, this aspect of our class can be further optimized due to limitations with span, which is one-dimensional, and has manual bounds checking. However, we can overcome these limitations via the use of C++23 mdspan (multi-dimensional span), which allows for automatic bound verification as well as the viewing of multidimensional arrays.

With this implementation, the above changes will instead appear as;

Matrix Storage Safe Access via mdspan

```
#include <iostream>
#include <cassert>
#include <span>
#include <experimental/mdspan>  // or <mdspan> in a fully compliant C++23 compiler

using std::size_t;
namespace stdex = std::experimental; // or std:: for C++23


// Class with Private data members
    class Matrix {
private:
    int M, N, LDA;
    // mdspan over double, 2D dynamic extents
    stdex::mdspan<double, stdex::extents<size_t, stdex::dynamic_extent, stdex::dynamic_extent>> data;

public:
    // Constructor for external data
    Matrix(int m, int n, int lda, double* ptr)
        : M(m), N(n), LDA(lda),
          data(ptr, m, n) {}  // mdspan automatically handles indexing

    // Safe element access
    double& at(int i, int j) {
        assert(i >= 0 && i < M && j >= 0 && j < N);
        return data(i, j);  // mdspan overloads operator() for indexing
    }
    const double& at(int i, int j) const {
        assert(i >= 0 && i < M && j >= 0 && j < N);
        return data(i, j);
    }

    int rows() const { return M; }
    int cols() const { return N; }
};
```

2.3 Setting Up for Greater Optimization

2.3.1 Experimenting via Simple Operations

Now that we have a more efficient foundation for storing our matrices, we can begin testing it by experimenting with different mathematical operations. Beginning with addition. We will update our class with the following method:

Matrix Addition Method

```
     Matrix add(const Matrix& B) const {
        assert(M==B.rows() && N==B.cols());
        Matrix C(M,N);
        for(int i=0;i<M;i++) for(int j=0;j<N;j++)
            C.at(i,j) = at(i,j) + B.at(i,j);
        return C;
    }

    void add_inplace(const Matrix& B, Matrix& C) const {
        assert(M==B.rows() && N==B.cols() && M==C.rows() && N==C.cols());
        for(int i=0;i<M;i++) for(int j=0;j<N;j++)
            C.at(i,j) = at(i,j) + B.at(i,j);
    }
```

We can test this method using the following matrices

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix} \quad Y = \begin{bmatrix} 10 & 11 & 12 \\ 11 & 12 & 13 \end{bmatrix}$$

Using these, our code outputs the following:

Reference vs Optimized Output

```
Testing add_inplace (53.6)...
    D after add_inplace (should be X+Y):
    11.000000 13.000000 15.000000
    13.000000 15.000000 17.000000
```

(See Appendix.A, Main)

Now that we have fleshed out our matrix class with more functionality, can we optimize what we currently have even further?

2.3.2 Debugging and Cache Optimization

Currently, we have the `at` method for potential debugging, however this alone is not efficient. If we make use of the preprocessor, we can introduce faster alternatives, that allow us to directly access data via

From 53.7. Matrix Storage, PAGE 543 of Introduction to Scientific Programming in C++23/Fortran2018 by Victor Eijkhout

```
    // gemm/lapack alloc.cpp
    auto get_double_data() {
        double *adata;
        adata = data.data();
        return adata;
};
```

Through the use of the `#define` preprocessor directive we can implement the following into our class,

53.7: macros and optimized indexing

```
// Release (non-DEBUG) leaf kernel will use these macros.
#ifdef DEBUG
  // When debugging we will use at() directly in leaf kernel.
#else
  #define A_AT_raw(ad, ldaA, i, j) (ad[(size_t)(i)*(size_t)(ldaA) + (size_t)(j)])
  #define B_AT_raw(bd, ldaB, i, j) (bd[(size_t)(i)*(size_t)(ldaB) + (size_t)(j)])
  #define C_AT_raw(cd, ldaC, i, j) (cd[(size_t)(i)*(size_t)(ldaC) + (size_t)(j)])
#endif
```

in addition to the leaf kernel:

Leaf kernel: C += A*B (no zeroing)

```
// This is the single place that performs the raw-pointer fast multiply. Higher-level
// algorithms (blocked/recursive) call this to accumulate partial products.
inline void MatMultLeafAdd(const Matrix& A, const Matrix& B, Matrix& C) {
    // dimension check: A.rows x A.cols  *  B.rows x B.cols  -> C.rows x C.cols,
    if (A.cols() != B.rows() || A.rows() != C.rows() || B.cols() != C.cols())
        throw std::runtime_error( MatMultLeafAdd: dimension mismatch );

#ifdef DEBUG
    // safe, but slower: use at()
    for (int i = 0; i < A.rows(); ++i) {
        for (int k = 0; k < A.cols(); ++k) {
            double aik = A.at(i, k);
            for (int j = 0; j < B.cols(); ++j)
                C.at(i, j) += aik * B.at(k, j);
        }
    }
#else
    const double* ad = A.raw_data();
    const double* bd = B.raw_data();
    double* cd = C.raw_data();
    int ldaA = A.lda(), ldaB = B.lda(), ldaC = C.lda();

    // IKJ ordering: for reuse of A row and B row
    for (int i = 0; i < A.rows(); ++i) {
        for (int k = 0; k < A.cols(); ++k) {
```

```
            double aik = A_AT_raw(ad, ldaA, i, k);
            // unroll or pointer bumps could be applied here in future
            for (int j = 0; j < B.cols(); ++j) {
                C_AT_raw(cd, ldaC, i, j) += aik * B_AT_raw(bd, ldaB, k, j);
            }
        }
    }
#endif
}
```

Which performs the multiplication on small sub-matrices. Our previously written **IKJ** loop order is used to improve spatial locality by keeping rows of **A** and **C** in the cache longer. In the release build, raw pointers and macros such as `A_AT_raw` allow us access to matrix elements directly, eliminating the overhead of function calls and bounds checking in the performance-critical inner loop. These work together to maximize cache efficiency and computational speed, allowing for further optimization.

Now that we have an optimized method for sub-matrix multiplication, we need to create methods to construct and use sub-matrices, which brings us to our next section.

2.4 Implementation of Sub-matrices and their Methods

2.4.1 Sub-matrix Construction and Verification

In order to support constructing actual sub-matrices (while aiming for decomposition in a 2×2 block form), we can write four methods, which we employ in our class as such

Sub-matrix Methods)

```
    Matrix TopLeft(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( TopLeft: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + 0 * LDA + 0;
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
    Matrix TopRight(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( TopRight: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + 0 * LDA + (N - n_sub);
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
    Matrix BotLeft(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( BotLeft: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + (M - m_sub) * LDA + 0;
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
```

```
    Matrix BotRight(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::
            runtime_error( BotRight: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + (M - m_sub) * LDA
            + (N - n_sub);
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
```

Which gives us the following output when tested with a short main function:
Which when checked with a small testing main function gives us the following output

```
Parent matrix A:
  1.00  2.00  3.00  4.00
  5.00  6.00  7.00  8.00
  9.00 10.00 11.00 12.00
 13.00 14.00 15.00 16.00

TopLeft:
  1.00  2.00
  5.00  6.00

TopRight:
  3.00  4.00
  7.00  8.00

BotLeft:
  9.00 10.00
 13.00 14.00

BotRight:
 11.00 12.00
 15.00 16.00

Parent after modifying TL:
100.00  2.00  3.00  4.00
  5.00  6.00  7.00  8.00
  9.00 10.00 11.00 12.00
 13.00 14.00 15.00 16.00

All submatrix tests passed!
```

(See Appendix.A, Sub-Matrix Test)

2.4.2 Implementation Through Various Methods

Now that we have successfully created usable sub-matrix methods and have verified that they function properly, we are able to use these methods to take an existing matrix operation, in our case, multiplication, and make it even more computationally efficient!

We can begin with a basic first multiplication algorithm which begins with the function

```
inline void MatMultLeafAdd(const Matrix& A, const Matrix& B, Matrix& C) {
    // function body...
}
```

that will be called to in the following function:

Base Optimized Matrix Multiplication Function

```
    inline void MatMultOptimized(const Matrix& A, const Matrix& B, Matrix& C) {
    if (A.cols() != B.rows() || A.rows() != C.rows() || B.cols() != C.cols())
        throw std::runtime_error( MatMultOptimized: dimension mismatch );
    C.zero();
    MatMultLeafAdd(A, B, C);
}
```

We can then take this a step further by introducing one level of blocking into our multiplication function which will use the previously mentioned 2 x 2 decomposition format.

Our new function and addition to our matrix class will appear as such

Optimized Matrix Multiplication Algorithm via One Level of Blocking

```
    // 53.3.1 BlockedMatMult (one-level blocking)
void Matrix::BlockedMatMult(const Matrix& B, Matrix& C, int BLOCK_SIZE) const {
    const Matrix& A = *this;
    int M_ = A.rows(), N_ = B.cols(), K_ = A.cols();
    if (M_ != C.rows() || N_ != C.cols() || K_ != B.rows())
        throw std::runtime_error( BlockedMatMult: dimension mismatch );

    C.zero();

    // Use submatrix views and call the leaf adder for each block product:
    for (int ii = 0; ii < M_; ii += BLOCK_SIZE) {
        int i_max = std::min(ii + BLOCK_SIZE, M_);
        for (int kk = 0; kk < K_; kk += BLOCK_SIZE) {
            int k_max = std::min(kk + BLOCK_SIZE, K_);
            for (int jj = 0; jj < N_; jj += BLOCK_SIZE) {
                int j_max = std::min(jj + BLOCK_SIZE, N_);

                // create external submatrix views: A_sub (i_max-ii x k_max-kk), B_sub (k_max-kk x j_max-jj), C_sub (i_max-ii x j_max-jj)
                double* A_ptr = const_cast<double*>(A.raw_data()) + ii * A.lda() + kk;
                double* B_ptr = const_cast<double*>(B.raw_data()) + kk * B.lda() + jj;
```

```
                    double* C_ptr = C.raw_data() + ii * C.lda() + jj;

                    Matrix A_sub(i_max - ii, k_max - kk, A.lda(), A_ptr);
                    Matrix B_sub(k_max - kk, j_max - jj, B.lda(), B_ptr);
                    Matrix C_sub(i_max - ii, j_max - jj, C.lda(), C_ptr);

                    // accumulate partial product into C_sub
                    MatMultLeafAdd(A_sub, B_sub, C_sub);
                }
            }
        }
}
```

The above function uses our sub-matrix construction and viewing methods (via pointers) to efficiently create 2x2 blocks which we will use to perform our full operation. After creating our sub-matrices, we call

```
MatMultLeafAdd(const Matrix& A, const Matrix& B, Matrix& C)
```

which will perform our permutated **IKJ** multiplication algorithm on our sub-matrix blocks, and since these blocks are small, they are likely to fit within CPU caches, making multiplication faster than if we had used the original naive IJK method over the entire parent matrix.

Below is a figure showing how these blocks will correspond to our greater C matrix

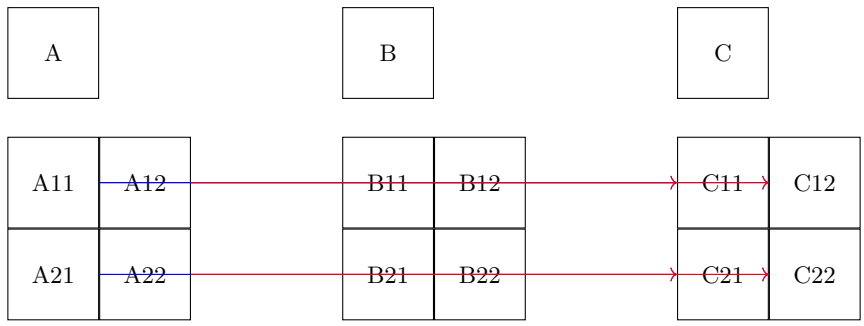

Figure 2: Blocked 2×2 matrix multiplication. Each C_{ij} is computed as the sum of two products of corresponding A and B blocks.

2.4.3 Optimization of Sub-Matrix Methods via Recursion

The final step in creating our optimized matrix multiplication method is to take our current blocking algorithm, and make it recursive. But why?

The first level of blocking partitions a matrix into blocks of a *fixed size*, which may still be too large to fit entirely in the CPU cache. By making the algorithm recursive, we retain all the advantages of the original block partitioning approach (spatial locality and optimized raw-pointer multiplication) while ensuring that blocks are small enough to fit in cache.

The algorithm will repeatedly partitions each block until it is small enough that the entire computation of;

```
MatMultLeafAdd(const Matrix& A, const Matrix& B, Matrix& C)
```

will happen within the cache, only loading each element one time, and reusing it for multiple computations. Ensuring faster and more memory efficient matrix multiplication.

The function and its definition will appear as follows:

Optimized Matrix Multiplication Algorithm via Recursive Blocking

```
void Matrix::RecursiveMatMult(const Matrix& B, Matrix& C, int THRESHOLD) const {
const Matrix& A = *this;
if (A.rows() != C.rows() || B.cols() != C.cols() || A.cols() != B.rows())
    throw std::runtime_error( RecursiveMatMult: dimension mismatch );

// base case: if small enough call leaf add (zero C first)
if (A.rows() <= THRESHOLD || A.cols() <= THRESHOLD || B.cols() <= THRESHOLD) {
    C.zero();
    MatMultLeafAdd(A, B, C);
    return;
}

// split sizes (floor halves)
int m2 = A.rows() / 2;
int k2 = A.cols() / 2;
int n2 = B.cols() / 2;

// build submatrix views (careful with odd sizes)
Matrix A11 = A.TopLeft(m2, k2);
Matrix A12 = A.TopRight(m2, A.cols() - k2);
Matrix A21 = A.BotLeft(A.rows() - m2, k2);
Matrix A22 = A.BotRight(A.rows() - m2, A.cols() - k2);

Matrix B11 = B.TopLeft(k2, n2);
Matrix B12 = B.TopRight(k2, B.cols() - n2);
Matrix B21 = B.BotLeft(B.rows() - k2, n2);
Matrix B22 = B.BotRight(B.rows() - k2, B.cols() - n2);

Matrix C11 = C.TopLeft(m2, n2);
Matrix C12 = C.TopRight(m2, C.cols() - n2);
Matrix C21 = C.BotLeft(C.rows() - m2, n2);
Matrix C22 = C.BotRight(C.rows() - m2, C.cols() - n2);

// temporaries for partial products (owned, with lda >= n)
Matrix T1(m2, n2, C11.lda());
Matrix T2(m2, n2, C11.lda());

// C11 += A11*B11 + A12*B21
T1.zero(); T2.zero();
A11.RecursiveMatMult(B11, T1, THRESHOLD);
A12.RecursiveMatMult(B21, T2, THRESHOLD);
```

```
    T1.add_inplace(T2, C11);

    // C12 += A11*B12 + A12*B22
    T1.zero(); T2.zero();
    A11.RecursiveMatMult(B12, T1, THRESHOLD);
    A12.RecursiveMatMult(B22, T2, THRESHOLD);
    T1.add_inplace(T2, C12);

    // C21 += A21*B11 + A22*B21
    T1.zero(); T2.zero();
    A21.RecursiveMatMult(B11, T1, THRESHOLD);
    A22.RecursiveMatMult(B21, T2, THRESHOLD);
    T1.add_inplace(T2, C21);

    // C22 += A21*B12 + A22*B22
    T1.zero(); T2.zero();
    A21.RecursiveMatMult(B12, T1, THRESHOLD);
    A22.RecursiveMatMult(B22, T2, THRESHOLD);
    T1.add_inplace(T2, C22);
}
```

Here, Temporary matrices `T1` and `T2` hold partial results for the 2×2 block products, allowing them to accumulate into the corresponding C_{ij} sub-matrix without overwriting intermediate results. Recursion continues splitting each block in half (floor division) until the sub-matrix size is below `THRESHOLD`, to which it will then call

```
MatMultLeafAdd(const Matrix& A, const Matrix& B, Matrix& C)
```

(for odd-sized matrices, the bottom and right blocks take the remaining rows or columns to ensure complete coverage) At the leaf level, `MatMultLeafAdd` performs the actual optimized multiplication, reusing the **IKJ** method we developed earlier. This recursive approach maximizes cache reuse, minimizes memory traffic, and ensures faster computation than either the naive or first-level blocked multiplication.

3 Testing Optimization: Performance Benchmarks Across Variations

Now that we have completed our optimized matrix class and multiplication method, we want to ensure that it functions the way it should.

3.1 Base Multiplication Correctness

First, we must verify that our optimized multiplication methods still output the same as a reference, naive **IJK** algorithm to ensure accuracy. When we compare each multiplication method with the following matrices

$$A_{\text{small}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \qquad B_{\text{small}} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

We *should* receive the following output

$$C_{\text{ref}} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

Is this the case?

Our algorithms give the following output:

```
C_opt (optimized IKJ):
 90.000000100.000000110.000000120.000000
202.000000228.000000254.000000280.000000
314.000000356.000000398.000000440.000000
426.000000484.000000542.000000600.000000
IKJ equals ref? YES
C_blocked (blocked):
 90.000000100.000000110.000000120.000000
202.000000228.000000254.000000280.000000
314.000000356.000000398.000000440.000000
426.000000484.000000542.000000600.000000
Blocked equals ref? YES
C_rec (recursive):
 90.000000100.000000110.000000120.000000
202.000000228.000000254.000000280.000000
314.000000356.000000398.000000440.000000
426.000000484.000000542.000000600.000000
Recursive equals ref? YES
```

(See Appendix.A, Main)

This demonstrates that our multiplication methods are indeed accurate to our original reference method, which lets us move on to performance testing with a sound mind.

3.2 Performance Optimization

In theory, due to memory allocation and reuse of data without copying or creating new elements, block partitioning, and its recursive variant should always be more efficient and competitive than their naive counterpart. However, is this the case?

We can test this by comparing the time it takes to complete operations. First, we include the `<chrono>` C++ library and then create a helper to help with our timing measurements, as seen below;

Time Measurement Helper

```
    template <typename F>
    double measure_time(F&& func) {
        using clock = std::chrono::high_resolution_clock;
        auto t1 = clock::now();
        func();
        auto t2 = clock::now();
        std::chrono::duration<double> elapsed = t2 - t1;
        return elapsed.count();
}
```

Then, by having a vector to give us various **nxn** sizes of a matrix in order to test each of these methods across a variety of examples, we then create loops for each algorithm (naive, blocked, recursive) and compare their outputs. Additionally, we can include a comparison to a computer's peak CPU performance. For this example, I will use my own CPU, a Intel Core Ultra 9. Which has its peak performance estimated at approximately 44 GFLOPs. We will compare this to the GFLOPs of each method by percentage metric.

We have tested matrices of the following sizes; **4x4**, **16x16**, **32x32**, **128x128**, **256x256**, **512x512**, **1028x1028**.

(Note: These sizes can be altered by simply changing the following vector in Main, given in Appendix.A)

```
 std::vector<int> sizes = {4, 16, 32, 128, 256, 512, 1028};
```

With these sizes, our tests give us the following outputs:

Performance Test Results

```
    Timing tests (naive / blocked / recursive)     various sizes

Size: 4 x 4
Naive:    1.031e-06 s, 0.124151 GFLOP/s, 0.282162% of peak
Blocked:  9.28e-07 s, 0.137931 GFLOP/s, 0.31348% of peak
Recursive:5.58e-07 s, 0.229391 GFLOP/s, 0.521342% of peak
Top-level MatMult correct? YES

Size: 16 x 16
Naive:    4.2649e-05 s, 0.19208 GFLOP/s, 0.436544% of peak
Blocked:  1.7246e-05 s, 0.475009 GFLOP/s, 1.07957% of peak
Recursive:1.6677e-05 s, 0.491215 GFLOP/s, 1.1164% of peak
Top-level MatMult correct? YES

Size: 32 x 32
Naive:    0.000399589 s, 0.164009 GFLOP/s, 0.372747% of peak
Blocked:  0.000121913 s, 0.537564 GFLOP/s, 1.22174% of peak
Recursive:0.00012187 s, 0.537753 GFLOP/s, 1.22217% of peak
Top-level MatMult correct? YES

Size: 128 x 128
```

```
Naive:    0.0204638 s, 0.204962 GFLOP/s, 0.465822% of peak
Blocked:  0.00759328 s, 0.552371 GFLOP/s, 1.25539% of peak
Recursive:0.00915657 s, 0.458065 GFLOP/s, 1.04106% of peak
Top-level MatMult correct? YES

Size: 256 x 256
Naive:    0.161267 s, 0.208068 GFLOP/s, 0.472882% of peak
Blocked:  0.0619074 s, 0.54201 GFLOP/s, 1.23184% of peak
Recursive:0.0763626 s, 0.439409 GFLOP/s, 0.998657% of peak
Top-level MatMult correct? YES

Size: 512 x 512
Naive:    1.4143 s, 0.189801 GFLOP/s, 0.431367% of peak
Blocked:  0.492096 s, 0.545494 GFLOP/s, 1.23976% of peak
Recursive:0.636985 s, 0.421416 GFLOP/s, 0.957763% of peak
Top-level MatMult correct? YES

Size: 1028 x 1028
Naive:    11.6185 s, 0.187007 GFLOP/s, 0.425017% of peak
Blocked:  3.9988 s, 0.54335 GFLOP/s, 1.23489% of peak
Recursive: skipped (size incompatible)
Top-level MatMult correct? YES

All tests completed.
```

(See Appendix.A, Main)

Above, you may notice a few things. First, it can be noted that the optimized algorithms are faster than the naive implementation. Additionally, this gap in speed grows as our matrix size increases due to lack of cache reuse and poor memory accesses.

You may also note that each of these algorithms uses a marginal amount of our peak processing power, causing us to be quite far from it. This is due to lack of memory access, especially for the naive algorithm, cache sizes limiting how much data can be reused efficiently, and because our algorithms could be optimized even further via multi-core parallelism.

This percentage does indeed depend on matrix size, as with small matrices, the total number of FLOPs is too low to fully saturate the CPU, but as matrices get larger, the recursive and blocked algorithms allow for better cache usage, but are still held back by memory bandwidth.

You may also note that the Recursive algorithm skips the **1028x1028** performance test. This is because the total memory allocated for this in our

```cpp
void Matrix::RecursiveMatMult(const Matrix& B, Matrix& C, int THRESHOLD) const {
// function body..
}
```

can exceed what our stack (given our original THRESHOLD set as 32) can handle (variance in threshold will be addressed in subsection NUMBR).

3.3 Discussion: Memory Reuse and Efficiency

A primary reason that the naive method is slower than the recursive is due to lack of data reuse.

in the following code;

Naive IJK Multiplication

```
        for(int i=0;i<4;i++)
            for(int j=0;j<4;j++){
                C_ijk.at(i,j)=0;
                for(int k=0;k<4;k++)
                    C_ijk.at(i,j) += A_small.at(i,k)*B_small.at(k,j);
        }
```

We access rows of `A` contiguously in memory, but columns of `B` are accessed in a strided manner. Meaning most elements of `B` are loaded from the main memory multiple times, and elements of `C` are only updated once per iteration. As a result, the computation is memory-bound, and cache reuse is minimal.

By comparison, the 2x2 block partitioning, recursive method, the matrices are partitioned into smaller blocks to fit our CPU cache. The leaf kernel then performs the multiplication on these blocks using raw-pointer access and a row-based access pattern. Each element of `A` and `B` is then loaded into cache once per block and reused for multiple computations of `C`. This reuses data and our cache efficiently and reduces memory bandwidth bottlenecks, resulting in faster computation, especially for large matrices.

In short, the recursive approach takes a large matrix, and cuts it into many small, cache-friendly subproblems, which is a key principle in high-performance linear algebra and its optimization.

3.4 Recursion and its Diminishing Returns

However, after a point, continuing to recurse will not give us any meaningful benefit once the products of recursion are fully contained within a cache

In our recursive matrix multiplication, a cutoff threshold is defined at which the algorithm stops recursing and switches to the leaf kernel (`MatMultLeafAdd`). Once the sub-matrices are small enough to fit within the CPU cache, recursion yields diminishing returns. This is because all the data required for leaf level multiplication is already in minimal cache memory, so subdividing matrices further does not reduce memory accesses any further.

Experimental results confirm this behavior. By varying the cutoff threshold, we observe that performance improves when approaching sizes matching the L1 or L2 cache, but further reducing the block size does not significantly increase speed. On my system, the L1 cache is about 32 KB per core, and the L2 cache is 512 KB per core. Any cutoff that produces blocks fitting within these caches maximizes data reuse and minimizes memory latency.

This demonstrates the connection between cache sizes and the optimal recursion cutoff: the algorithm should recurse until each block fits in the cache, but no further.

If we update our performance tests to attempt various thresholds, we can see this difference in speed in the following output:

Performance Test Results Including Threshold Variance

```
Timing tests (naive / blocked / recursive with thresholds)     various
    sizes

Matrix size: 4 x 4
```

```
Naive:    1.064e-06 s, 0.120301 GFLOP/s, 0.273411% of peak
Blocked:  1.031e-06 s, 0.124151 GFLOP/s, 0.282162% of peak

Matrix size: 16 x 16
Recursive (TH=8): 2.8771e-05 s, 0.284731 GFLOP/s, 0.647116% of peak
Recursive (TH=16): 1.6762e-05 s, 0.488724 GFLOP/s, 1.11074% of peak
Naive:    4.5223e-05 s, 0.181147 GFLOP/s, 0.411697% of peak
Blocked:  1.7175e-05 s, 0.476972 GFLOP/s, 1.08403% of peak

Matrix size: 32 x 32
Recursive (TH=8): 0.000251485 s, 0.260596 GFLOP/s, 0.592264% of peak
Recursive (TH=16): 0.000160477 s, 0.408383 GFLOP/s, 0.928142% of peak
Recursive (TH=32): 0.000120864 s, 0.542229 GFLOP/s, 1.23234% of peak
Naive:    0.000328994 s, 0.199201 GFLOP/s, 0.45273% of peak
Blocked:  0.000121618 s, 0.538868 GFLOP/s, 1.2247% of peak
```

(See Appendix.A, Main)

Additionally, as said previously where our largest matrix was skipped for our recursive algorithm, with a varying threshold, we are able to output and receive the following:

```
Matrix size: 1024 x 1024
Recursive (TH=8): 1.65232 s, 1.29968 GFLOP/s, 2.95382% of peak
Recursive (TH=16): 0.729819 s, 2.94249 GFLOP/s, 6.68748% of peak
Recursive (TH=32): 0.553316 s, 3.88112 GFLOP/s, 8.82072% of peak
Recursive (TH=64): 0.329555 s, 6.51631 GFLOP/s, 14.8098% of peak
Naive:    5.1692 s, 0.415438 GFLOP/s, 0.944178% of peak
Blocked:  0.21375 s, 10.0467 GFLOP/s, 22.8334% of peak
```

(See Appendix.A, Main)

4 Conclusion

When completing this project, much of the optimization was based on memory allocation and reuse. The reason the naive approach falls behind with matrix multiplication is due to it copying elements rather than reusing existing ones. We solved this by implementing a block partitioning algorithm in order to split larger matrices and use their parts to compute their products. This way, we reuse the smaller blocks and sum them without having to copy individual elements. We took this a step further by making the algorithm recursive so that it will split matrices until the blocks are of a size that will guaranteed fit into our CPU caches allowing for faster, more memory efficient computation. Then to confirm the theory, we tested the time measurements and CPU performance of each, and proved that our optimizations were indeed better than the naive at a variety of matrix sizes and different thresholds (for recursive).

As for personal challenges with this project, I had the most difficulty ensuring consistency of data and pointers across such a large class. What I had spent the most amount of time on was ensuring that the blocked and recursive methods were actually optimized, as for awhile they took even longer than the naive algorithm at a variety of sizes. This was due to me not fully exploiting cache and data reuse. Previously, the functions would make copies of their T1 and T2 matrices on top of the algorithms they

were already completing, significantly padding their run time, making it seem as if naive would win every time. I addressed this by making sure the debugging method and leaf kernel written for Exercise 53.7. were fully used and present in each of the methods. Following this and a few minor data consistency tweaks, the output now matched the theory at a variety of levels, and even demonstrated that the gap in computation speed between naive and optimized grew as matrices grew larger.

Additionally, it may be noted that the final class code in Appendix.A does not make use of mdspan. This is due to compilation errors and difficulty when trying to run this code within Stampede3. Upon further research and several attempts at alternative ways to compile mdspan, I was informed that it was simply incompatible on Stampede3s compiler. However, the way mdspan would have theoretically been used can be found above in section 2.

Ultimately, this project provided me a good challenge and helped me learn more about memory and cache usage by different computational methods and how it may affect performance for better or for worse. While the final class is not as optimized as it could be (which could be done by adding multi-core vector parallelism or CBlas implementation) it is significantly faster than the original naive IJK routine, which was a primary goal in this project.

5 Appendix

5.1 A: Code Snippets

Headers

```cpp
#include <iostream>
#include <vector>
#include <span>
#include <cassert>
#include <iomanip>
#include <cmath>
#include <chrono>
#include <stdexcept>
#include <algorithm>
```

Full Matrix Class

```cpp
    class Matrix {
    int M{}, N{}, LDA{};                    // rows, cols, leading dimension (stride)
    std::vector<double> owner;             // owns memory when non-empty
    std::span<double> data;                // view into memory (owner or external)
    bool is_external = false;

public:
    // Owned matrix: contiguous rows with LDA == n (default)
    Matrix(int m = 0, int n = 0)
        : M(m), N(n), LDA(n), owner(m * n), data(owner.data(), owner.size()) {}

    // Owned with explicit lda (lda >= n)
    Matrix(int m, int n, int lda)
        : M(m), N(n), LDA(lda), owner(m * lda), data(owner.data(), owner.size()) {
        if (lda < n) throw std::invalid_argument( lda must be >= n );
    }

    // External view constructor (does NOT take ownership)
    Matrix(int m, int n, int lda, double* ptr)
        : M(m), N(n), LDA(lda), owner(), data(ptr, m * lda), is_external(true) {
        if (lda < n) throw std::invalid_argument( lda must be >= n );
        if (ptr == nullptr && m * lda > 0) throw std::invalid_argument( null pointer for external data );
    }

    // Factory names (book vs intuitive)
```

```
    static Matrix External(int m, int n, int lda, double* ptr) { return Matrix(m, n, lda, ptr); }
    static Matrix External_book(int m, int lda, int n, double* ptr) { return Matrix(m, n, lda, ptr); }

    // Accessors (53.4)
    int rows() const { return M; }
    int cols() const { return N; }
    int lda()  const { return LDA; }

    // raw_data: const and non-const overloads
    double* raw_data() { return data.data(); }
    const double* raw_data() const { return data.data(); }

    // get_double_data() analogue
    double* get_double_data() { return raw_data(); }
    const double* get_double_data() const { return raw_data(); }

    // bounds-checked element access
    double& at(int i, int j) {
        if (i < 0 || i >= M || j < 0 || j >= N) throw std::out_of_range(Matrix::at index out of range );
        return data[i * LDA + j];
    }
    const double& at(int i, int j) const {
        if (i < 0 || i >= M || j < 0 || j >= N) throw std::out_of_range(Matrix::at index out of range );
        return data[i * LDA + j];
    }

    // zero logical MxN region
    void zero() {
        for (int i = 0; i < M; ++i)
            for (int j = 0; j < N; ++j)
                data[i * LDA + j] = 0.0;
    }

    // printing
    void print(std::ostream& os = std::cout) const {
        os << std::fixed << std::setprecision(6);
        for (int i = 0; i < M; ++i) {
            for (int j = 0; j < N; ++j)
                os << std::setw(10) << at(i, j);
            os <<  \n ;
        }
        os << std::defaultfloat;
    }

    // approx equal
```

```
    bool approx_equal(const Matrix& B, double tol = 1e-9) const {
        if (rows() != B.rows() || cols() != B.cols()) return false;
        for (int i = 0; i < M; ++i)
            for (int j = 0; j < N; ++j)
                if (std::abs(at(i, j) - B.at(i, j)) > tol) return false;
        return true;
    }

    // add_inplace: dest += (*this) + other (53.6)
    // Used in recursive algorithm: T1.add_inplace(T2, C11) -> C11 += T1 + T2
    void add_inplace(const Matrix& other, Matrix& dest) const {
        if (M != other.M || N != other.N || M != dest.M || N != dest.N)
            throw std::runtime_error( add_inplace: dimension mismatch );
        for (int i = 0; i < M; ++i)
            for (int j = 0; j < N; ++j)
                dest.data[i * dest.LDA + j] += this->data[i * LDA + j] + other.data[i * other.LDA + j];
    }

    // Submatrix views (53.8) (TopLeft, TopRight, BotLeft, BotRight)
    // These create external views pointing into the same storage.
    Matrix TopLeft(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( TopLeft: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + 0 * LDA + 0;
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
    Matrix TopRight(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( TopRight: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + 0 * LDA + (N - n_sub);
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
    Matrix BotLeft(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( BotLeft: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + (M - m_sub) * LDA + 0;
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
    Matrix BotRight(int m_sub, int n_sub) const {
        if (m_sub < 0 || n_sub < 0 || m_sub > M || n_sub > N) throw std::runtime_error( BotRight: bad dims );
        double* ptr = const_cast<double*>(raw_data()) + (M - m_sub) * LDA + (N - n_sub);
        return Matrix::External(m_sub, n_sub, LDA, ptr);
    }
```

```

    // ---- Declarations (definitions below) ----
    void MatMult(const Matrix& B, Matrix& C) const;        // C = A*B (overwrite)
    Matrix MatMult(const Matrix& B) const;                 // return A*B

    void BlockedMatMult(const Matrix& B, Matrix& C, int BLOCK_SIZE = 64) const;
    void RecursiveMatMult(const Matrix& B, Matrix& C, int THRESHOLD = 32) const;

    static Matrix ReferenceMatMult(const Matrix& A, const Matrix& B);
};

// 53.7: macros and optimized indexing
// Release (non-DEBUG) leaf kernel will use these macros.
#ifdef DEBUG
  // When debugging we will use at() directly in leaf kernel.
#else
  #define A_AT_raw(ad, ldaA, i, j) (ad[(size_t)(i)*(size_t)(ldaA) + (size_t)(j)])
  #define B_AT_raw(bd, ldaB, i, j) (bd[(size_t)(i)*(size_t)(ldaB) + (size_t)(j)])
  #define C_AT_raw(cd, ldaC, i, j) (cd[(size_t)(i)*(size_t)(ldaC) + (size_t)(j)])
#endif

// Leaf kernel: C += A*B (no zeroing)
// This is the single place that performs the raw-pointer fast multiply. Higher-level
// algorithms (blocked/recursive) call this to accumulate partial products.
inline void MatMultLeafAdd(const Matrix& A, const Matrix& B, Matrix& C) {
    // dimension check: A.rows x A.cols  *  B.rows x B.cols  -> C.rows x C.cols,
    if (A.cols() != B.rows() || A.rows() != C.rows() || B.cols() != C.cols())
        throw std::runtime_error( MatMultLeafAdd: dimension mismatch );

#ifdef DEBUG
    // safe, but slower: use at()
    for (int i = 0; i < A.rows(); ++i) {
        for (int k = 0; k < A.cols(); ++k) {
            double aik = A.at(i, k);
            for (int j = 0; j < B.cols(); ++j)
                C.at(i, j) += aik * B.at(k, j);
        }
    }
#else
```

```
    const double* ad = A.raw_data();
    const double* bd = B.raw_data();
    double* cd = C.raw_data();
    int ldaA = A.lda(), ldaB = B.lda(), ldaC = C.lda();

    // IKJ ordering: for reuse of A row and B row
    for (int i = 0; i < A.rows(); ++i) {
        for (int k = 0; k < A.cols(); ++k) {
            double aik = A_AT_raw(ad, ldaA, i, k);
            // unroll or pointer bumps could be applied here in future
            for (int j = 0; j < B.cols(); ++j) {
                C_AT_raw(cd, ldaC, i, j) += aik * B_AT_raw(bd, ldaB, k, j);
            }
        }
    }
#endif
}

// Simple wrapper: C = A*B (zeros C then accumulates)
inline void MatMultOptimized(const Matrix& A, const Matrix& B, Matrix& C) {
    if (A.cols() != B.rows() || A.rows() != C.rows() || B.cols() != C.cols())
        throw std::runtime_error( MatMultOptimized: dimension mismatch );
    C.zero();
    MatMultLeafAdd(A, B, C);
}

// Matrix method definitions
void Matrix::MatMult(const Matrix& B, Matrix& C) const {
    if (rows() != C.rows() || B.cols() != C.cols() || cols() != B.rows())
        throw std::runtime_error( MatMult: dimension mismatch );
    MatMultOptimized(*this, B, C);
}

Matrix Matrix::MatMult(const Matrix& B) const {
    Matrix C(rows(), B.cols());
    MatMult(B, C);
    return C;
}

Matrix Matrix::ReferenceMatMult(const Matrix& A, const Matrix& B) {
    if (A.cols() != B.rows()) throw std::runtime_error( ReferenceMatMult: dimension mismatch );
    Matrix C(A.rows(), B.cols());
    C.zero();
    int M = A.rows(), N = B.cols(), K = A.cols();
    for (int i = 0; i < M; ++i)
```

```
        for (int j = 0; j < N; ++j) {
            double s = 0.0;
            for (int k = 0; k < K; ++k) s += A.at(i, k) * B.at(k, j);
            C.at(i, j) = s;
        }
    return C;
}

// 53.3.1 BlockedMatMult (one-level blocking)
void Matrix::BlockedMatMult(const Matrix& B, Matrix& C, int BLOCK_SIZE) const {
    const Matrix& A = *this;
    int M_ = A.rows(), N_ = B.cols(), K_ = A.cols();
    if (M_ != C.rows() || N_ != C.cols() || K_ != B.rows())
        throw std::runtime_error( BlockedMatMult: dimension mismatch );

    C.zero();

    // Use submatrix views and call the leaf adder for each block product:
    for (int ii = 0; ii < M_; ii += BLOCK_SIZE) {
        int i_max = std::min(ii + BLOCK_SIZE, M_);
        for (int kk = 0; kk < K_; kk += BLOCK_SIZE) {
            int k_max = std::min(kk + BLOCK_SIZE, K_);
            for (int jj = 0; jj < N_; jj += BLOCK_SIZE) {
                int j_max = std::min(jj + BLOCK_SIZE, N_);

                // create external submatrix views: A_sub (i_max-ii x k_max-kk), B_sub (k_max-kk x j_max-jj), C_sub (i_max-ii x j_max-jj)
                double* A_ptr = const_cast<double*>(A.raw_data()) + ii * A.lda() + kk;
                double* B_ptr = const_cast<double*>(B.raw_data()) + kk * B.lda() + jj;
                double* C_ptr = C.raw_data() + ii * C.lda() + jj;

                Matrix A_sub(i_max - ii, k_max - kk, A.lda(), A_ptr);
                Matrix B_sub(k_max - kk, j_max - jj, B.lda(), B_ptr);
                Matrix C_sub(i_max - ii, j_max - jj, C.lda(), C_ptr);

                // accumulate partial product into C_sub
                MatMultLeafAdd(A_sub, B_sub, C_sub);
            }
        }
    }
}

// 53.9 RecursiveMatMult (2x2 block recursion)
// Uses TopLeft/TopRight/BotLeft/BotRight for submatrix views and uses MatMultLeafAdd at leaves.
```

```
void Matrix::RecursiveMatMult(const Matrix& B, Matrix& C, int THRESHOLD) const {
    const Matrix& A = *this;
    if (A.rows() != C.rows() || B.cols() != C.cols() || A.cols() != B.rows())
        throw std::runtime_error( RecursiveMatMult: dimension mismatch );

    // base case: if small enough call leaf add (zero C first)
    if (A.rows() <= THRESHOLD || A.cols() <= THRESHOLD || B.cols() <= THRESHOLD) {
        C.zero();
        MatMultLeafAdd(A, B, C);
        return;
    }

    // split sizes (floor halves)
    int m2 = A.rows() / 2;
    int k2 = A.cols() / 2;
    int n2 = B.cols() / 2;

    // build submatrix views (careful with odd sizes)
    Matrix A11 = A.TopLeft(m2, k2);
    Matrix A12 = A.TopRight(m2, A.cols() - k2);
    Matrix A21 = A.BotLeft(A.rows() - m2, k2);
    Matrix A22 = A.BotRight(A.rows() - m2, A.cols() - k2);

    Matrix B11 = B.TopLeft(k2, n2);
    Matrix B12 = B.TopRight(k2, B.cols() - n2);
    Matrix B21 = B.BotLeft(B.rows() - k2, n2);
    Matrix B22 = B.BotRight(B.rows() - k2, B.cols() - n2);

    Matrix C11 = C.TopLeft(m2, n2);
    Matrix C12 = C.TopRight(m2, C.cols() - n2);
    Matrix C21 = C.BotLeft(C.rows() - m2, n2);
    Matrix C22 = C.BotRight(C.rows() - m2, C.cols() - n2);

    // temporaries for partial products (owned, with lda >= n)
    Matrix T1(m2, n2, C11.lda());
    Matrix T2(m2, n2, C11.lda());

    // C11 += A11*B11 + A12*B21
    T1.zero(); T2.zero();
    A11.RecursiveMatMult(B11, T1, THRESHOLD);
    A12.RecursiveMatMult(B21, T2, THRESHOLD);
    T1.add_inplace(T2, C11);

    // C12 += A11*B12 + A12*B22
    T1.zero(); T2.zero();
    A11.RecursiveMatMult(B12, T1, THRESHOLD);
```

```
    A12.RecursiveMatMult(B22, T2, THRESHOLD);
    T1.add_inplace(T2, C12);

    // C21 += A21*B11 + A22*B21
    T1.zero(); T2.zero();
    A21.RecursiveMatMult(B11, T1, THRESHOLD);
    A22.RecursiveMatMult(B21, T2, THRESHOLD);
    T1.add_inplace(T2, C21);

    // C22 += A21*B12 + A22*B22
    T1.zero(); T2.zero();
    A21.RecursiveMatMult(B12, T1, THRESHOLD);
    A22.RecursiveMatMult(B22, T2, THRESHOLD);
    T1.add_inplace(T2, C22);
}
```

Time Measurement Helper

```
    template <typename F>
    double measure_time(F&& func) {
        using clock = std::chrono::high_resolution_clock;
        auto t1 = clock::now();
        func();
        auto t2 = clock::now();
        std::chrono::duration<double> elapsed = t2 - t1;
        return elapsed.count();
}
```

Main Function Used to Conduct Method Tests

```
    //  Main: small correctness tests + timing
int main() {
    std::cout <<  Running small correctness tests (4x4)...\n ;

    // A_small
    Matrix A_small(4, 4);
    for (int i = 0; i < 4; ++i) for (int j = 0; j < 4; ++j) A_small.at(i,
        j) = i * 4 + j + 1;

    double raw[16] = {1,2,3,4, 5,6,7,8, 9,10,11,12, 13,14,15,16};
    Matrix B_small = Matrix::External_book(4, 4, 4, raw);

    std::cout <<  A_small:\n ; A_small.print();
    std::cout <<  B_small:\n ; B_small.print();

    // Reference naive IJK
    Matrix C_ref = Matrix::ReferenceMatMult(A_small, B_small);
```

```

    // MatMult (top-level: zero + leaf add)
    Matrix C_opt(4,4);
    A_small.MatMult(B_small, C_opt);

    std::cout << "C_ref (reference):\n" ; C_ref.print();
    std::cout << "C_opt (optimized IKJ):\n" ; C_opt.print();
    std::cout << "IKJ equals ref? " << (C_ref.approx_equal(C_opt) ? "YES" : "NO") << "\n" ;

    // Blocked MatMult
    Matrix C_blocked(4,4);
    A_small.BlockedMatMult(B_small, C_blocked, 2);
    std::cout << "C_blocked (blocked):\n" ; C_blocked.print();
    std::cout << "Blocked equals ref? " << (C_ref.approx_equal(C_blocked) ? "YES" : "NO") << "\n" ;

    // Recursive MatMult
    Matrix C_rec(4,4);
    A_small.RecursiveMatMult(B_small, C_rec, 2);
    std::cout << "C_rec (recursive):\n" ; C_rec.print();
    std::cout << "Recursive equals ref? " << (C_ref.approx_equal(C_rec) ? "YES" : "NO") << "\n" ;

    // 53.6 test: add_inplace for different LDA
    std::cout << "\nTesting add_inplace (53.6)...\n" ;
    Matrix X(2,3,5); // m=2, n=3, lda=5 (padding per row)
    Matrix Y(2,3,6); // lda different
    Matrix D(2,3);   // destination
    X.zero(); Y.zero(); D.zero();
    for (int i=0;i<2;i++) for (int j=0;j<3;j++){ X.at(i,j) = 1.0 + i + j; Y.at(i,j) = 10.0 + i + j; }
    Matrix T1 = X; Matrix T2 = Y;
    T1.add_inplace(T2, D);
    std::cout << "D after add_inplace (should be X+Y):\n" ; D.print();

// Timing / performance with multiple recursive thresholds
std::cout << "\nTiming tests (naive / blocked / recursive with thresholds) various sizes\n" ;
const double peak_GFLOPs = 44.0; // estimated peak

// Include small, moderate, and large matrices
std::vector<int> sizes = {4, 16, 32, 128, 256, 512};
std::vector<int> thresholds = {8, 16, 32, 64}; // recursive cutoff points

for (int N : sizes) {
    std::cout << "\nMatrix size: " << N << " x " << N << "\n" ;
    Matrix A(N,N), B(N,N), C(N,N);

```

```
    // Initialize matrices with simple values
    for (int i=0;i<N;i++)
        for (int j=0;j<N;j++){
            A.at(i,j) = (i+j)%100 + 1;
            B.at(i,j) = (i-j)%100 + 1;
        }

    double flops = 2.0 * N * (double)N * (double)N;

    // --- Naive triple-loop ---
    C.zero();
    double t_naive = measure_time([&](){
        for (int i=0;i<N;i++)
            for (int j=0;j<N;j++){
                double s = 0.0;
                for (int k=0;k<N;k++) s += A.at(i,k) * B.at(k,j);
                C.at(i,j) = s;
            }
    });
    double g_naive = flops / (t_naive * 1e9);

    // --- Blocked ---
    C.zero();
    double t_block = measure_time([&](){ A.BlockedMatMult(B, C, 64); });
    double g_block = flops / (t_block * 1e9);

    // --- Recursive for multiple thresholds ---
    for (int TH : thresholds) {
        if (N < TH) continue; // skip if matrix smaller than threshold

        C.zero();
        double t_rec = measure_time([&](){ A.RecursiveMatMult(B, C, TH);
            });
        double g_rec = flops / (t_rec * 1e9);

        std::cout <<  Recursive (TH=  << TH <<  ):
                  << t_rec <<   s,
                  << g_rec <<   GFLOP/s,
                  << (g_rec/peak_GFLOPs*100.0) <<  % of peak\n ;
    }

    std::cout <<  Naive:      << t_naive <<   s,   << g_naive <<   GFLOP/s
        ,
              << (g_naive/peak_GFLOPs*100.0) <<  % of peak\n ;
    std::cout <<  Blocked:    << t_block <<   s,   << g_block <<   GFLOP/s
        ,
              << (g_block/peak_GFLOPs*100.0) <<  % of peak\n ;
}
```

```

std::cout <<  \nAll tests completed.\n ;
return 0;


}
```

Sub-Matrix Test

```
#include <iostream>
#include <vector>
#include <span>
#include <cassert>
#include <iomanip>
#include <stdexcept>

class Matrix {
    int M, N, LDA;
    std::vector<double> owner;
    std::span<double> data;

public:
    Matrix(int m, int n) : M(m), N(n), LDA(n), owner(m * n), data(owner.
        data(), owner.size()) {}

    Matrix(int m, int n, int lda, double* ptr) : M(m), N(n), LDA(lda),
        owner(), data(ptr, m*lda) {}

    double& at(int i, int j) { return data[i * LDA + j]; }
    const double& at(int i, int j) const { return data[i * LDA + j]; }

    void print() const {
        std::cout << std::fixed << std::setprecision(2);
        for (int i = 0; i < M; ++i) {
            for (int j = 0; j < N; ++j) std::cout << std::setw(6) << at(i,
                j);
            std::cout <<  \n ;
        }
    }

    // Submatrix methods
    Matrix TopLeft(int m_sub, int n_sub) const {
        double* ptr = const_cast<double*>(data.data());
        return Matrix(m_sub, n_sub, LDA, ptr);
    }
    Matrix TopRight(int m_sub, int n_sub) const {
        double* ptr = const_cast<double*>(data.data()) + (N - n_sub);
        return Matrix(m_sub, n_sub, LDA, ptr);
```

```
    }
    Matrix BotLeft(int m_sub, int n_sub) const {
        double* ptr = const_cast<double*>(data.data()) + (M - m_sub)*LDA;
        return Matrix(m_sub, n_sub, LDA, ptr);
    }
    Matrix BotRight(int m_sub, int n_sub) const {
        double* ptr = const_cast<double*>(data.data()) + (M - m_sub)*LDA +
            (N - n_sub);
        return Matrix(m_sub, n_sub, LDA, ptr);
    }
};

int main() {
    // 1. Create a parent matrix 4x4
    Matrix A(4,4);
    int val = 1;
    for (int i=0;i<4;i++) for (int j=0;j<4;j++) A.at(i,j) = val++;

    std::cout << "Parent matrix A:\n" ; A.print();

    // 2. Extract submatrices
    Matrix TL = A.TopLeft(2,2);
    Matrix TR = A.TopRight(2,2);
    Matrix BL = A.BotLeft(2,2);
    Matrix BR = A.BotRight(2,2);

    // 3. Print submatrices
    std::cout << "\nTopLeft:\n" ; TL.print();
    std::cout << "\nTopRight:\n" ; TR.print();
    std::cout << "\nBotLeft:\n" ; BL.print();
    std::cout << "\nBotRight:\n" ; BR.print();

    // 4. Modify TL and verify parent updated
    TL.at(0,0) = 100;
    std::cout << "\nParent after modifying TL:\n" ; A.print();

    // 5. Assertions
    assert(TL.at(0,0) == 100);
    assert(A.at(0,0) == 100);
    assert(TR.at(0,0) == 3);
    assert(BL.at(0,0) == 9);
    assert(BR.at(0,0) == 11);

    std::cout << "\nAll submatrix tests passed!\n" ;
    return 0;
}
```